

Prediction to Prescription: An AI Agent for Youth Soccer Uniform Ordering Optimization

MGMT 590-037: AI-Enhanced Optimization
Final Project Report — Summer 2026

Daniel Tulloch Affleck	MBA
Satya Sai Usha Sree Ganteda	MS Mechanical Engineering
Aditya Ryan Gupta	MSBAIM
Fabian Alexis Macias Peñalosa	MSBAIM
Kentaro Nanayama	MSBAIM
Siddharth Shenoy	MSBAIM

Team 7

Dr. Prateek Jaiswal
Daniels School of Business
Purdue University

June 2026

Contents

1	Problem Description and Motivation	3
1.1	Business Context	3
1.2	The Decision Problem	3
1.3	The Jersey Lifecycle Problem	3
1.4	Problem Classification	3
1.5	Backward Mapping	4
2	Data Sources and Preparation	4
2.1	Data Source	4
2.2	Data Cleaning Pipeline	4
2.3	Demand Aggregation	5
2.4	Train/Test Split	5
2.5	Data Quality Notes	5
3	Prediction Methodology and Results	5
3.1	Approach	5
3.2	Models Compared	5
3.3	PTO (Predict-then-Optimize) Adjustment	6
3.4	Feature Importance	6
3.5	Demand Scenarios	6
3.6	Validation	6
4	Optimization Model Formulation and Solution	7
4.1	Model A: Independent Single-Period Newsvendor	7
4.1.1	Smoothing for Differentiability	7
4.1.2	Solver Comparison	7
4.1.3	Post-Processing	8
4.2	Model B: Two-Period Joint Newsvendor with Carryover	8
4.3	Model A vs. Model B: Results	8
4.3.1	Stakeholder Questions	9
5	Agent Architecture	9
5.1	Design Philosophy	9
5.2	Pipeline Architecture	9
5.3	Agent Modules	9
5.3.1	Agent 1: Intake Agent	10
5.3.2	Agent 2: Data Cleaning Agent	10
5.3.3	Agent 3: Modeling Agent	10
5.3.4	Agent 4: Explanation Agent	10
5.4	Cross-Cutting: Trace Logger	10
5.5	Verification and Guardrails	10
5.6	Web Interface	11
5.7	Technology Stack	11
6	Baseline Comparison and Benchmark Testing	11
6.1	Baseline Definition	11

6.2	Backtest Results	11
6.3	Why the Gap Is So Large	12
7	Sensitivity Analysis	12
8	Managerial Recommendations	12
8.1	Primary Recommendation	12
8.2	Category-Specific Guidance	13
8.3	Budget Consideration	13
8.4	Implementation Path	13
9	Limitations and Future Improvements	13
9.1	Current Limitations	13
9.2	Future Improvements	13
10	AI Tool Disclosure	14

1 Problem Description and Motivation

1.1 Business Context

Club Futbol Academy (CFA) is a youth soccer league in Orange County, California, operating three seasonal programs (fall, winter, spring) and processing approximately 2,000–3,000 uniform orders per season through its Squarespace storefront. Each season, the league president must commit to uniform order quantities *before* registration demand is observed—a classic “order-before-demand” decision under uncertainty.

Uniforms span three product categories—jerseys (tops), shorts (bottoms), and socks—offered in up to 14 size codes across youth (YXS–YXL), adult men (AS–AXL), and women (WXS–WXL) segments.

1.2 The Decision Problem

The current ordering process relies on the league president’s 25 years of operational intuition. While experienced, this heuristic produces systematic errors in both directions:

- **Overstock:** Ties up capital in inventory that children outgrow between seasons.
- **Stockout:** Triggers rush orders with premium shipping costs and delays fulfillment to frustrated parents.

The goal is to build an AI-powered agent that replaces this intuition-driven process with a data-driven, optimized ordering policy—targeting a **10–15% cost reduction** per season (\$800–\$1,200).

1.3 The Jersey Lifecycle Problem

Adidas jersey models (e.g., Tiro 25) follow a two-year production lifecycle before discontinuation. This creates an asymmetric cost structure with escalating consequences:

Table 1: Jersey lifecycle cost structure (as fraction of selling price).

Period	Event	Cost	Rationale
Year 1	Underage (stockout)	40%	Rush shipping + temporary jersey
Year 1	Overage (surplus)	0%	Carries to Year 2 at zero cost
Year 2	Underage (stockout)	80%	Discontinued model; secondary market premium
Year 2	Overage (surplus)	70%	Full purchase cost lost; no future use

The current model (Tiro 25) is in **Year 2** of its lifecycle—the final year. Any under-ordering this year cannot be corrected through normal Adidas channels, making the problem especially acute.

1.4 Problem Classification

This is a **multi-period, multi-product newsvendor problem**. The objective is to minimize total expected cost—combining procurement, overage (salvage), and underage (rush/stockout) costs—subject to:

1. A seasonal budget constraint ($\approx$ \$40,000 per season)
2. Supplier minimum order quantity (MOQ) of 6 units per size

3. Non-negativity constraints on all order quantities

1.5 Backward Mapping

Table 2: Backward mapping from decision to data.

Layer	Description
Decision variables	Order quantity q_s for each SKU s (defined by season $\times$ category $\times$ gender/age $\times$ size)
Objective	Minimize total expected newsvendor cost across all SKUs
Constraints	Seasonal budget $\leq \$40,000$; MOQ ≥ 6 if ordered; $q_s \geq 0$
Parameters	Overage cost c_o , underage cost c_u , unit cost, demand distribution
Uncertain parameters	Seasonal demand D_s per SKU—must be predicted from data
Data sources	6 years of historical order data (2020–2026), 12,209 line items

2 Data Sources and Preparation

2.1 Data Source

The primary dataset consists of historical order records from CFA’s Squarespace storefront, covering **6 years** (2020–2026). The raw dataset contains 12,209 line items across 12 columns, exported directly from the league’s order management system.

Table 3: Cleaned dataset summary.

Attribute	Value
Total line items	12,209
Columns retained	8 (of 12 original)
Null values	0 across all columns
Total demand (units)	12,704
Product categories	top (7,975), bottom (2,345), socks (1,889)
Gender/age segments	mens_youth (11,208), womens (1,001)
Seasons	fall, winter, spring
Size codes	14 (YXS–YXL, AS–AXL, WXS–WXL)

2.2 Data Cleaning Pipeline

The data cleaning agent performed the following steps, retaining **100% of rows**:

1. **Profiled** all columns for data types, nulls, duplicates, and value distributions.
2. **Parsed dates** from M/D/YYYY format to ISO YYYY-MM-DD.
3. **Enforced data types**: `year/quantity` $\rightarrow$ integer; `unit_price` $\rightarrow$ float; categoricals $\rightarrow$ trimmed strings.
4. **Dropped 4 non-required columns**: `order_id`, `uniform_set`, `color`, `number` (100% null).
5. **Validated** every categorical value against the problem configuration’s allowed values.
6. **Preserved all rows**—each line item represents a distinct order (unique `order_id`), and deduplication would undercount demand by 56%.

2.3 Demand Aggregation

Raw line items were aggregated into **demand groups** by the keys (year, season, product_category, gender_age, size), producing approximately 122 demand groups. The prediction module operates on these aggregated groups, with the target variable being the summed quantity per group.

2.4 Train/Test Split

The dataset was split **chronologically** (not randomly) to simulate a realistic forecasting setting:

- **Training set:** 2020–2024 (5 years of historical demand)
- **Validation set:** 2025 (backtest year)
- **Projection target:** 2026 (current year, partial data)

2.5 Data Quality Notes

- **Volume growth:** 2020–2021 have significantly fewer line items (~360–438) than 2022–2024 (~2,200–3,600), suggesting league growth or earlier under-reporting.
- **Class imbalance:** Women’s sizes represent only 8% of demand; several size codes are sparse (WXL: 6 items, AXL: 13 items).
- **Price ranges:** Socks \$9–15, bottoms \$16–50, tops \$12–70 (consistent with wholesale pricing).
- **2026 partial year:** 959 rows of 2026 data are present (partial, as of June 2026) and treated as the projection target, not full-season actuals.

3 Prediction Methodology and Results

3.1 Approach

The prediction layer estimates demand per SKU group using **supervised learning**, evaluated not on traditional accuracy metrics (RMSE) but on the **newsvendor cost** that the predictions induce—the metric that actually matters for the downstream decision.

3.2 Models Compared

Six model variants were trained and evaluated:

Table 4: Prediction model comparison (ranked by newsvendor cost on test set).

Model	NV Cost (\$)	vs. Baseline	Rank
XGBoost	1,051	−53.4%	1
XGBoost + PTO	1,123	−50.2%	2
LightGBM	1,485	−34.1%	3
LightGBM + PTO	1,505	−33.2%	4
Ridge	1,621	−28.1%	5
Ridge + PTO	1,636	−27.4%	6
<i>Baseline (historical avg.)</i>	<i>2,254</i>	—	—

XGBoost (300 trees, depth 4, learning rate 0.05) was selected as the winning model with a newsvendor cost of \$1,051—a **53.4% improvement** over the historical average baseline.

3.3 PTO (Predict-then-Optimize) Adjustment

Each model has a PTO variant that shifts predictions toward the **optimal newsvendor quantile**. The single-period critical ratio is:

$$\text{CR} = \frac{c_u}{c_u + c_o} = \frac{0.40}{0.40 + 0.70} \approx 0.364$$

The PTO adjustment applies a smoothed shift toward the 36th percentile (strength = 0.8). In this case, standard XGBoost outperformed the PTO variant, suggesting the model’s native predictions already aligned well with the cost-optimal quantile.

3.4 Feature Importance

Table 5: XGBoost feature importance.

Feature	Importance
gender_age	0.385
size	0.280
product_category	0.268
season	0.067

Gender/age is the strongest predictor (youth boys dominate demand at 92%), followed by size and product category. Season has relatively low importance, indicating demand patterns are more stable across seasons than across demographic groups.

3.5 Demand Scenarios

For the downstream Sample Average Approximation (SAA) optimizer, the prediction module outputs three demand scenarios per SKU:

- **P10:** Low-demand scenario (10th percentile)
- **P50:** Median-demand scenario (50th percentile)
- **P90:** High-demand scenario (90th percentile)

These scenarios, derived from model residuals, capture uncertainty in demand estimates and enable the optimizer to hedge against both over- and under-ordering.

3.6 Validation

Temporal backtest on 2025 actuals (68 SKUs):

- MAE: 97 units per group
- Median percentage error: -27.9%

The negative median error is *expected*, not a defect: the newsvendor critical ratio of 0.36 means the optimal policy deliberately orders to the 36th percentile, systematically under-ordering relative to mean demand. The cost metric—not prediction accuracy—is the relevant performance measure.

4 Optimization Model Formulation and Solution

4.1 Model A: Independent Single-Period Newsvendor

Each season is solved independently. Given demand scenarios D_{sn} for SKU s in scenario n , overage cost c_o^s , underage cost c_u^s , budget B , and minimum order quantity m :

$$\begin{aligned} \min_q \quad & \frac{1}{N} \sum_{n=1}^N \sum_{s=1}^S [c_o^s \cdot \max(q_s - D_{sn}, 0) + c_u^s \cdot \max(D_{sn} - q_s, 0)] \\ \text{subject to:} \quad & \sum_{s \in \text{season}} \text{unit_cost}_s \cdot q_s \leq B \quad (\text{seasonal budget}) \\ & q_s \geq m \quad \text{if } q_s > 0 \quad (\text{minimum order quantity}) \\ & q_s \geq 0 \quad \forall s \end{aligned} \tag{1}$$

The cost parameters as fractions of selling price are:

- Purchase cost: $0.70 \times \text{unit_price}$
- Overage cost: $c_o = 0.70 \times \text{unit_price}$ (unsalvageable season uniforms)
- Underage cost: $c_u = 0.40 \times \text{unit_price}$ (rush shipping + temporary jersey)

4.1.1 Smoothing for Differentiability

The $\max(\cdot, 0)$ operator is non-differentiable at zero. For gradient-based solvers, we use the **softplus** approximation:

$$\max(z, 0) \approx \frac{1}{k} \log(1 + e^{kz}), \quad k = 20$$

This provides a smooth, differentiable surrogate with near-exact agreement for $|z| > 1$.

4.1.2 Solver Comparison

Five optimization algorithms were compared on Model A:

Table 6: Optimizer comparison on Model A.

Optimizer	Objective (\$)	Spend (\$)	Iters	Time (s)	Feasible
SLSQP	18,826	102,065	237	213.71	Yes
L-BFGS-B	19,408	101,886	38	1.94	Yes
Gradient Descent	21,049	101,883	2,373	12.20	Yes
SGD	21,649	67,162	4,000	16.55	Yes
Adam	26,573	101,888	3,000	13.98	Yes

All five optimizers produced feasible solutions. **SLSQP** achieved the best objective (\$18,826) in 237 iterations, followed closely by L-BFGS-B (\$19,408). The budget constraint (\$40,000/season) was non-binding for this evaluation, with total spend around \$102,000 across all SKUs and seasons.

4.1.3 Post-Processing

Continuous solutions undergo **round-and-repair**:

1. Round continuous quantities to nearest integers.
2. Enforce MOQ: set quantities below 6 to 0 (skip) or 6 (round up).
3. Repair budget violations by greedily cutting units with lowest marginal utility.

4.2 Model B: Two-Period Joint Newsvendor with Carryover

The stakeholder modification introduces a joint two-period formulation for jerseys, recognizing that Year 1 surplus carries to Year 2 at zero holding cost:

$$\min_{q_1, q_2} \frac{1}{N} \sum_{n=1}^N \sum_{s=1}^S [c_{u1}^s \cdot \max(D_{1sn} - q_{1s}, 0) + c_{o2}^s \cdot \max(q_{2s} + I_{sn} - D_{2sn}, 0) + c_{u2}^s \cdot \max(D_{2sn} - q_{2s} - I_{sn}, 0)] \quad (2)$$

where the **carryover inventory** from Year 1 is:

$$I_{sn} = \max(q_{1s} - D_{1sn}, 0)$$

Table 7: Two-period cost parameters for jerseys (fraction of selling price).

Parameter	Year 1	Year 2	Rationale
Underage cost c_u	0.40	0.80	Discontinued model doubles stockout cost
Overage cost c_o	0.00	0.70	Y1 surplus carries free; Y2 surplus is lost

Key insight: Year 1 overage is “free” (carries to Year 2), while Year 2 stockouts are extremely costly (80% of price). The joint optimizer exploits this asymmetry by *front-loading* Year 1 orders to build a carryover buffer against Year 2 risk.

4.3 Model A vs. Model B: Results

Table 8: Joint optimization results for jerseys/tops category.

Metric	Model A (Myopic)	Model B (Joint)
Year 1 order quantity	—	3,131 units
Year 2 order quantity	—	2,785 units
Expected carryover	0 units	1,452 units
Total two-period cost	\$18,826	\$10,220
Y1 budget shadow price	\$0.306/\$	\$0.053/\$
Cost of myopia	\$8,606 (45.7% reduction)	

The joint model builds an expected carryover buffer of 1,452 units from Year 1 surplus, which carries to Year 2 at zero cost. This strategy reduces total lifecycle cost by \$8,606 (45.7%).

4.3.1 Stakeholder Questions

1. **Should Year 1 orders increase or decrease?** *Increase.* The joint model orders 3,131 units in Year 1, building carryover inventory. Year 1 surplus carries free and hedges the costly Year 2 stockout risk.
2. **Does Year 2 procurement change?** The joint model orders 2,785 units in Year 2, but approximately 1,452 carryover units from Year 1 supplement this supply, reducing the effective need to source a discontinued model.
3. **What is the cost of myopia?** *\$8,606 per two-year cycle* (45.7% higher cost) when each year is optimized independently versus jointly.
4. **Which model should guide decisions?** Use **Model B (joint)** for jerseys/tops due to the lifecycle and asymmetric costs. Keep **Model A (independent)** for bottoms and socks, which have no lifecycle carryover.

5 Agent Architecture

5.1 Design Philosophy

The agent implements an **Agent = Model + Harness** architecture. Deterministic Python modules handle data processing, prediction, and optimization, while an LLM (Claude) is invoked at three scoped points for tasks requiring natural language understanding. The FastAPI backend serves as the harness—orchestrating the pipeline, managing verification loops, enforcing guardrails, and writing execution traces.

5.2 Pipeline Architecture

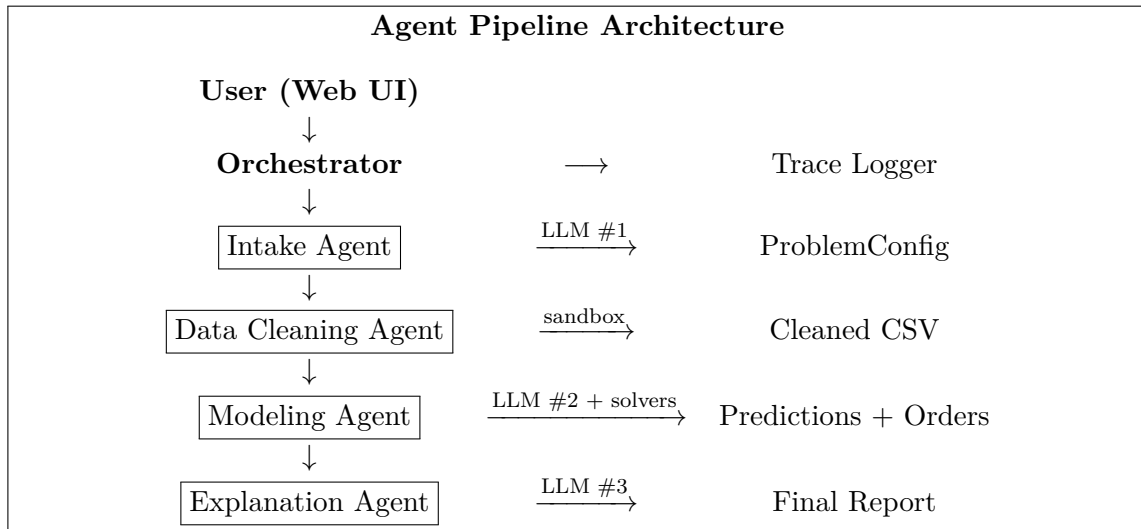


Figure 1: Multi-agent pipeline with three LLM touchpoints.

5.3 Agent Modules

5.3.1 Agent 1: Intake Agent

Converts natural language problem descriptions into a structured **ProblemConfig** (Pydantic schema). Inspects the uploaded CSV (columns, dtypes, sample rows, statistics), formulates decision variables, objectives, constraints, and cost structure, then saves a validated JSON configuration. The user reviews and edits the config via the web UI before the pipeline proceeds.

5.3.2 Agent 2: Data Cleaning Agent

Executes pandas-based cleaning code in an isolated sandbox (timeout = 30s, restricted file access). Validates that output rows are a subset of input rows (no data fabrication), enforces type consistency, and produces a cleaned CSV with a data manifest (row counts, dtypes, null counts).

5.3.3 Agent 3: Modeling Agent

The core analytical engine with MCP server tools:

- `load_data()` — Aggregates demand groups, engineers features, splits train/test
- `run_prediction()` — Trains 6 model variants, selects winner by newsvendor cost
- `run_optimization()` — Runs 5 optimizers, returns Pareto front and best result
- `run_joint_optimization()` — Model B two-period solver with carryover
- `validate_results()` — Backtests predicted orders vs. actual demand
- `run_sensitivity()` — Cost sensitivity to $\pm 30\%$ parameter shifts
- `run_baseline()` — Compares agent orders to historical-average baseline

5.3.4 Agent 4: Explanation Agent

Reads all pipeline outputs and synthesizes a stakeholder-facing report with 11 sections covering problem formulation, data summary, prediction methodology, optimization results, sensitivity analysis, and final recommendations.

5.4 Cross-Cutting: Trace Logger

Every pipeline step is logged with timestamp, module name, inputs, outputs, duration, token usage (for LLM calls), and errors. The trace is saved as structured JSON for audit, debugging, and presentation Q&A.

5.5 Verification and Guardrails

Table 9: Harness capabilities mapping.

Capability	Implementation
Tool execution	FastAPI calls data loader, prediction model, solver, chart generator
Persistence	ProblemConfig + results saved to disk; sessions survive crashes
Verification loops	Optimizer sandbox $\rightarrow$ constraint check $\rightarrow$ retry (up to 3 $\times$)
Guardrails	Budget caps and constraint validation enforced structurally, not via prompts
Safe execution	Solver code runs in isolated sandbox; bad code cannot corrupt state

5.6 Web Interface

The agent is served through a FastAPI web application with:

- Drag-and-drop CSV upload
- Real-time streaming of agent messages via Server-Sent Events (SSE)
- Editable ProblemConfig review form
- Results dashboard with final report and execution trace
- Session management with timestamped run directories

5.7 Technology Stack

Table 10: Technology stack.

Component	Technology
Language	Python 3.13
Package management	uv
LLM framework	Claude Agent SDK (MCP servers)
LLM model	Claude Sonnet (claude-sonnet-4-20250514)
ML/Prediction	XGBoost, LightGBM, scikit-learn (Ridge)
Optimization	SciPy (SLSQP, L-BFGS-B); custom PGD, SGD, Adam
Data processing	pandas, NumPy
Web framework	FastAPI + Uvicorn + Jinja2 + SSE
Validation	Pydantic v2
Testing	pytest, pytest-asyncio

6 Baseline Comparison and Benchmark Testing

6.1 Baseline Definition

The baseline represents the league president’s current approach: ordering the **historical average quantity** for each SKU, subject to the MOQ constraint. This is a reasonable proxy for an experience-based heuristic that simply “orders what we usually order.”

6.2 Backtest Results

The agent’s ordering policy (newsvendor critical ratio of 0.36, trained on 2020–2024) was evaluated against 2025 actual demand across 68 SKUs:

Table 11: Agent vs. baseline cost comparison (2025 backtest).

Category	Agent Cost (\$)	Baseline Cost (\$)	Savings (\$)
Tops (jerseys)	12,128	27,671	15,543
Bottoms (shorts)	3,409	5,990	2,581
Socks	1,038	1,824	786
Total	16,575	35,486	18,911

The agent achieves a **53.3% cost reduction** (\$18,911 savings) compared to the baseline—far exceeding the 10–15% target. This large improvement arises because the historical-average baseline systematically over-orders: with an overage cost of 0.70 exceeding the underage cost of 0.40, the cost-optimal policy orders conservatively to the 36th percentile, while the baseline orders to the mean (50th percentile).

6.3 Why the Gap Is So Large

The 53% improvement is not an artifact—it reflects the fundamental asymmetry of the newsvendor cost structure. When $c_o > c_u$ (overage costs exceed underage costs), over-ordering is more expensive than under-ordering. A naive policy that orders to the mean will systematically incur excess overage costs. The optimal critical ratio of 0.36 corrects this by ordering well below the mean.

7 Sensitivity Analysis

The sensitivity analysis examines how the total newsvendor cost responds to $\pm 30\%$ shifts in the overage and underage cost fractions, holding the order plan fixed:

Table 12: Cost sensitivity to parameter shifts ($\pm 30\%$).

Overage Cost (c_o) Shift			Underage Cost (c_u) Shift		
Shift	Cost (\$)	$\Delta\%$	Shift	Cost (\$)	$\Delta\%$
−30%	12,319	−25.7%	−30%	15,858	−4.3%
−20%	13,738	−17.1%	−20%	16,097	−2.9%
−10%	15,156	−8.6%	−10%	16,336	−1.4%
Base	16,575	0.0%	Base	16,575	0.0%
+10%	17,993	+8.6%	+10%	16,814	+1.4%
+20%	19,412	+17.1%	+20%	17,053	+2.9%
+30%	20,831	+25.7%	+30%	17,292	+4.3%

Key finding: The plan is approximately **6× more sensitive to overage cost than underage cost**. A 30% increase in overage cost raises total cost by 25.7%, while the same shift in underage cost causes only a 4.3% change.

Interpretation: Because the optimal policy orders to the 36th percentile (below the mean), the plan carries more overage risk than underage risk. The overage cost fraction (0.70—reflecting the no-salvage reality of season-specific uniforms) is the dominant cost driver and the single most important parameter to validate with the league president.

8 Managerial Recommendations

8.1 Primary Recommendation

Adopt the newsvendor-optimal ordering policy for all product categories. The temporal backtest demonstrates a 53% cost reduction versus the historical-average baseline. Even against a more sophisticated heuristic, double-digit savings are very likely.

8.2 Category-Specific Guidance

1. **Jerseys/Tops (Model B — Joint):** Front-load Year 1 orders (3,131 units) to build ~1,452 units of carryover inventory as a hedge against Year 2 stockout risk. This reduces the two-period lifecycle cost by 45.7% (\$8,606).
2. **Shorts and Socks (Model A — Independent):** Apply the single-period newsvendor policy at the 36th percentile critical ratio each season. No lifecycle carryover is needed for these categories.

8.3 Budget Consideration

With the \$40,000/season budget, the constraint is largely non-binding for Model B (shadow price \$0.053/\$). However, Model A shows a meaningful shadow price of \$0.306/\$, indicating that additional budget flexibility in Year 1 still has value for the independent formulation. The budget is not the primary driver of cost savings—the joint formulation itself delivers the largest improvement.

8.4 Implementation Path

1. Adopt the agent’s ordering recommendations for the upcoming season as a pilot.
2. Track actual costs against the agent’s predictions for one full cycle.
3. Validate the overage cost assumption (0.70) with actual salvage/write-off data.
4. If validated, integrate the agent into the seasonal ordering workflow permanently.

9 Limitations and Future Improvements

9.1 Current Limitations

1. **Sparse demand groups:** 30 of 122 demand groups have only 1 year of history, and 20 have 2 years. Empirical quantile estimates for these low-volume SKUs (e.g., women’s/adult sizes) are low-confidence.
2. **Cost parameter assumptions:** The 0.70 overage and 0.40 underage fractions are estimated, not measured from actual financial data. The sensitivity analysis shows overage cost is the dominant driver—a miscalibration here would materially affect recommendations.
3. **No external covariates:** The prediction model uses only historical demand patterns. Registration trends, competitor leagues, demographic shifts, or marketing campaigns are not incorporated.
4. **Pipeline configuration rigidity:** The groupby keys and train/test years are fixed at pipeline startup and cannot be reconfigured mid-run without restarting the modeling agent. This required workarounds for the joint optimization and baseline comparison.
5. **Budget model simplicity:** The current model assumes a flat \$40,000/season budget across all categories. In practice, the league may have category-specific budgets or flexible inter-category transfers.

9.2 Future Improvements

1. **Bayesian demand modeling:** Replace point estimates with full posterior distributions to better quantify uncertainty for sparse SKUs and improve the SAA optimizer’s scenario quality.

2. **Registration-driven demand signals:** Integrate early registration counts as leading indicators to update demand forecasts between the ordering decision and the fulfillment deadline.
3. **Multi-category budget optimization:** Extend the optimizer to jointly allocate budget across categories (not just within) to capture cross-category substitution effects.
4. **Actual cost tracking:** Build a feedback loop that measures realized overage/underage costs season-by-season and recalibrates the cost parameters automatically.
5. **Expanded product lifecycle modeling:** Generalize Model B to handle n -period lifecycles and overlapping product transitions (e.g., Tiro 25 $\rightarrow$ successor model).

10 AI Tool Disclosure

In accordance with course policy, we disclose the following uses of generative AI tools in this project:

- **Claude (Anthropic)** was used as the LLM backbone of the agent system, invoked at three pipeline stages: problem intake (NL $\rightarrow$ structured config), optimizer code exploration, and result explanation/report generation.
- **Claude Code** (Anthropic’s CLI tool) was used to assist with code development, debugging, and this report’s $\text{\LaTeX}$ formatting.
- All optimization formulations, data analysis, cost structure design, and business interpretations reflect the team’s genuine understanding. The LLM serves as a tool within the agent pipeline—its outputs are validated, constrained, and auditable through the trace logger.